AUTOMATIONS + PEPITOS

Me automaticé la mañana. Después se me fue de las manos.

Empezó con un chat, una tarde de julio. Hoy son cinco rutinas y cuatro agentes que trabajan mientras yo decido. Lo que funcionó, lo que rompí, y cómo armar el tuyo.

- 30 minutos + preguntas
- Orca en vivo
- Seis capítulos, una historia real
- Todo corre en mi máquina



Los Pepitos. Arte de Fran.

PASO 0 · LA HERRAMIENTA

Todo lo que hago empieza en Orca.

Una app de escritorio, gratis y open source, para correr agentes en paralelo. Ahí viven mis sesiones y mis automatizaciones.

SESIONES	Una por tarea: un ticket, una review, un mensaje. Cada una abre en su propio workspace y no se mezclan.
WORKSPACE = GIT WORKTREE	Una copia del repo en otra carpeta, con su propia rama, sin volver a clonar. Cinco tareas abiertas, cero pisadas. Es una función de git, no de Orca.
AUTOMATIONS	Un prompt con un horario. Corre en una sesión nueva, sin nadie mirando, en mi máquina: laptop apagada, rutina que no corre. Jira, Slack, GitHub y las notas de reuniones están conectados.

→ Demo: un vistazo a Orca, 45 segundos



onorca.dev

orca.build/docs/model/worktrees

git-scm.com/docs/git-worktree

CAPÍTULO 1 · JULIO

Estaba desorganizado.

Mucho trabajo en el proyecto, poco orden, y cosas que se me pasaban. Cada mañana arrancaba reconstruyendo qué tenía que hacer.

01

Cuatro lugares

Jira, Slack, GitHub y las notas de las reuniones. El contexto estaba repartido y yo lo juntaba a mano.

02

Cosas que se caían

PRs que esperaban review días porque lo pedía «al equipo». Hilos que quedaban semanas sin respuesta. Preguntas que nunca contestaba.

03

La media hora más cara

La primera media hora del día, cuando más criterio tenés, se iba en juntar contexto en vez de en decidir.

CAPÍTULO 2

02

Automaticé la mañana.

Con todo lo que necesitaba: las cuatro fuentes, un archivo por día, y preguntas que no bloquean. Funcionó re bien.

PRIMERO EL DESTINO

Esto me esperaba hoy a las 8.

FOCO	**Marcar lista la PR de permisos**	Y pedir review a alguien con nombre: la anterior tardó cuatro días por pedirla «al equipo».
FOCO	**Tickets esperando QA**	Mandar la lista completa: quince, ocho con más de tres semanas.
RÁPIDO	**El hilo de hace 19 días**	Contestar. Un «sí / no» alcanza.
ESPERA	**Respuesta del tech lead**	La debe desde el jueves. Sin eso, la PR cambia de forma.
PREGUNTA	**¿Estos dos tickets son el mismo bug?**	(repetida, sexto día)

→ Demo: el archivo de hoy, tal cual lo escribió la rutina

CUATRO FUENTES	Jira, GitHub, Slack y las notas de las reuniones, leídas antes de que yo abra la laptop.
UN ARCHIVO POR DÍA	En un repo git, en español, con links.
NADIE LO ESCRIBIÓ	Lo escribió una rutina a las 08:00, sin que nadie la mirara. El de hoy está en Orca: lo vemos ahora.

CÓMO SE HACE · SIN CÓDIGO

Una automatización son tres cosas. Ninguna es código.

.MD	**daily-goals.md: cada mañana hábil, mirar mis fuentes y escribir qué hago hoy, en orden.**	187 líneas
PASOS	**1. El «Para mañana» de ayer · 2. Archivar las reuniones · 3. Las cuatro fuentes, siempre · 4. Buscar duplicados**	cuatro pasos
SALIDA	**Foco · Rápidos · Libres · Bloqueadores · KPIs · Relojes · Puntos · Reuniones · Preguntas**	nueve secciones
REGLAS	**Primero lo que destraba a otro, lo que lleva días esperando, y las preguntas que no pudo resolver sola.**	prioridad

Todo el conocimiento está en el documento. Cualquiera en esta sala puede escribir ese documento.

1 · UN DOCUMENTO CON LOS PASOS

En tu idioma: qué fuentes mirar, qué priorizar, dónde escribir. Lo escribí con Claude una tarde de julio y lo corrijo cada vez que falla.

2 · UN HORARIO EN ORCA

Días hábiles a las 08:00, sesión nueva, en mi máquina. Sin infra. Primera corrida real: lunes 20 de julio.

3 · UNA LÍNEA DE PROMPT

«Run daily-goals.md exactly as written. Run unattended; ask only if something is unclear.» Eso es todo lo que sabe Orca.

PARA TU PRIMERA RUTINA

Mis tres reglas para la primera rutina.

01

Una sola, la de la mañana

No automatices tu trabajo. Automatizá la media hora que te lo prepara. Cuando esa funcione, la siguiente la vas a ver sola.

02

Que escriba, no que hable

Un archivo por día que vos leés, en un repo con historia. Nada de mensajes que se pierden ni resúmenes que nadie vuelve a abrir.

03

Que pregunte sin bloquear

Cuando no sabe, anota la pregunta y sigue con su mejor suposición. Vos contestás cuando podés. Ella insiste hasta que lo hagas.

> **Y una que aprendí a los golpes: que pueda terminar sin hacer nada. Una rutina que necesita producir algo cada vez, inventa trabajo.**

MIS AUTOMATIZACIONES · TODAS

Hoy son cinco. Y el día se cierra en un ciclo.

08:00 · el plan → **11:30 · el standup** → **17:00 · el cierre** → **mañana 08:00 lee «Para mañana»**

08:00

El plan del día

Cuatro fuentes, un archivo: foco, rápidos, bloqueadores, relojes y preguntas.

11:30

El standup

Ayer / Hoy / Bloqueadores en diez líneas. Solo queda lo que otra persona destraba hoy.

17:00

El cierre

Hecho / No hecho / Diferente, y el «Para mañana» que la mañana siguiente lee primero.

CADA HORA

Bugbuster

Lee la lista de feedback en Slack. Bug: ticket. Mejora: pregunta en el hilo. La vemos en un minuto.

VIERNES

El coder aprende

Lee los comentarios de la semana en mis PRs y los vuelve reglas. Esa va en la segunda parte.

Cinco documentos, un archivo por día en git. Cuatro escriben y yo leo; la única que publica bajo mi nombre corre con un modelo fuerte y, ante la duda, no publica.

UNA HISTORIA

Mi automatización me persigue.

PREGUNTA	**¿Estos dos tickets son el mismo bug?**	(repetida, sexto día)
PREGUNTA	**¿«Ready for Release» = «PR lista»?**	Aunque no haya mergeado. (repetida)
FOCO 4	**Decidir el par de tickets**	Sexto día. Si es el mismo bug, cerrar uno; si no, dejar escrito qué lo distingue.

NO INVENTA	Cuando no puede decidir algo, lo anota en «Preguntas» y sigue con su mejor suposición.
NO SE OLVIDA	Cada mañana la vuelve a poner, con el contador de días. Es más insistente que yo.
Y TIENE RAZÓN	Tardé seis días en contestar una pregunta de un minuto. Sin la rutina, nunca.

Las preguntas no bloquean. Tampoco desaparecen.

PEPITO BUGBUSTER 🏴‍☠️💸 · CADA HORA, EN HORARIO DE TRABAJO

Del feedback en Slack al ticket con PR.

LEE

La lista de feedback

Una lista en Slack es la fuente de verdad: cada fila tiene su hilo y su estado.

→

TRIAGEA

Bug o mejora

Bug: ticket vía ticket-machine y un intento de PR draft. Mejora: preguntas en el hilo hasta que Producto apruebe.

→

NOTIFICA

Corto y en inglés

Actualiza el estado, responde en el hilo, anota en el diario.

Nació de un pedido real

Empezó con un mensaje directo. La primera corrida real fue al día siguiente.

Publica bajo mi nombre

Por eso corre con un modelo fuerte: decide con criterio o no publica. La mayoría de las corridas termina sin escribir nada, y eso es lo esperado.

LO QUE APRENDÍ A MIRAR

¿Qué automatizar? El test del toil, y después la cuenta.

Manual

Si nadie aprieta teclas, no pasa.

Repetitivo

Lo hiciste la semana pasada y lo vas a hacer la que viene.

Automatizable

Una máquina lo haría igual. Nadie lo describió todavía.

Táctico

Lo dispara algo de afuera: un mensaje, un ticket, una alerta.

Sin valor duradero

Cuando terminás, todo quedó igual que antes.

Escala con el trabajo

Más tickets, más clientes, más reuniones: más de esto.

LA CUENTA

5 min por día vale 6 días · 30 min por día vale 5 semanas · 1 h por semana vale 10 días

Cuánto vale invertir en automatizar algo que vas a seguir haciendo cinco años. Mi mañana era la fila de las cinco semanas y la armé en una tarde. Basado en xkcd 1205.

MATIZ

No todos los minutos valen igual

La media hora de la mañana vale más que la de las cuatro de la tarde: es cuando tenés criterio, y la gastás juntando contexto. Por eso empecé por esa.

- Tres de seis: candidata
- Cinco de seis: ¿por qué lo seguís haciendo vos?
- Google SRE Book, «Eliminating Toil» · sre.google/sre-book/eliminating-toil

CAPÍTULO 3

03

Gasté muchos créditos.

La rutina, el código, las reviews: todo en el mismo modelo caro. La ventana de Claude se me agotaba al toque.

CAPÍTULO 3 · LA VENTANA DE CRÉDITOS

Nos fuimos de tema con el gasto. A mitad de la mañana, a esperar.

La rutina, el código, las reviews: todo en el mismo modelo caro. La ventana de créditos de Claude se agotaba al toque y, hasta que se renovaba, a esperar.

01

Todo en el modelo caro

El mismo modelo para leer un diff que para decidir qué hacer con él. Lo mecánico costaba lo mismo que lo importante.

02

Contexto de más

Cada sesión releía Jira, Slack y GitHub desde cero, y los diffs entraban enteros: cuarenta archivos para sacar tres líneas.

03

La espera

A mitad de la mañana ya estaba mirando el reloj, esperando que se renovara la ventana. El trabajo, parado.

Había que hacer lo mismo con muchos menos tokens. Sin perder criterio.

CAPÍTULO 4

04

Aparecieron los Pepitos.

Para delegar el trabajo. Y ahí se me fue de las manos. A mí me dicen Pepe; los agentes son extensiones mías: Pepitos. Las galletitas son una coincidencia feliz.

CAPÍTULO 4 · EL PRESUPUESTO DE TOKENS

La salida fue delegar.

Cada trabajo al modelo más barato que lo haga bien, con límites claros. Los baratos leen y ejecutan; el caro decide. Así nacieron los Pepitos.

Haiku · lee
scout + librarian

→

Sonnet · ejecuta
coder + ticket-machine

→

Opus/Fable · decide
la sesión principal, y yo

> **«Para razonar y pensar, Opus/Fable; para lo sencillo, Sonnet; y hasta Haiku si conviene.»**

EL EQUIPO · UN ROL Y UN MODELO CADA UNO

Todos los Pepitos, en una pantalla.

Un Pepito es un agente al que le delego un trabajo, y lo ejecuta de forma correcta y limitada. La sesión principal decide; cuatro subagentes hacen lo mecánico; dos skills orquestan; cinco rutinas corren solas.

SONNET

coder

Escribe o arregla código a partir de una spec precisa, con mis estándares. Nunca commitea, ni decide producto, ni agranda el scope.

HAIKU

scout

Lee GitHub y CI: checks, diffs, comentarios, hilos. Devuelve archivo, línea y snippet. Nunca interpreta, nunca toca código.

HAIKU

librarian

Lee y escribe la base de conocimiento: contexto por cliente, handoffs, el archivo del día. Nunca toca código.

SONNET

ticket-machine

Escribe tickets cortos que un humano quiera leer, después de buscar duplicados. Nunca decide producto, nunca saltea la búsqueda.

El «nunca» es la parte más importante de cada definición: ejecutan bien porque tienen límites.

CAPÍTULO 5

05

Los flujos que siempre repetía.

Implementar un ticket y revisar una PR eran procesos marcados: siempre los mismos pasos. Así nacieron Pepito implement-ticket y Pepito review-pr.

FLUJO 1 · UN COMANDO `/pepito:implement-ticket ABC-1234`

Del ticket al PR draft en verde.

Fase 0 · preguntas → **Branch** → **Código · coder** → **Tests · lint · build** → **PR draft · self-review** → **Handoff · notas para retomar**

00

Fase 0 manda

«Hacé muchas preguntas hasta estar 100% seguro. No empieces hasta que estemos alineados.» Así lo pedí; así funciona. Plan aprobado antes de tocar código.

01

Scope discipline

Se entrega el plan aprobado y nada más. Lo que sobra se nombra en una línea y se revierte antes de commitear.

02

Sin acceso a la base

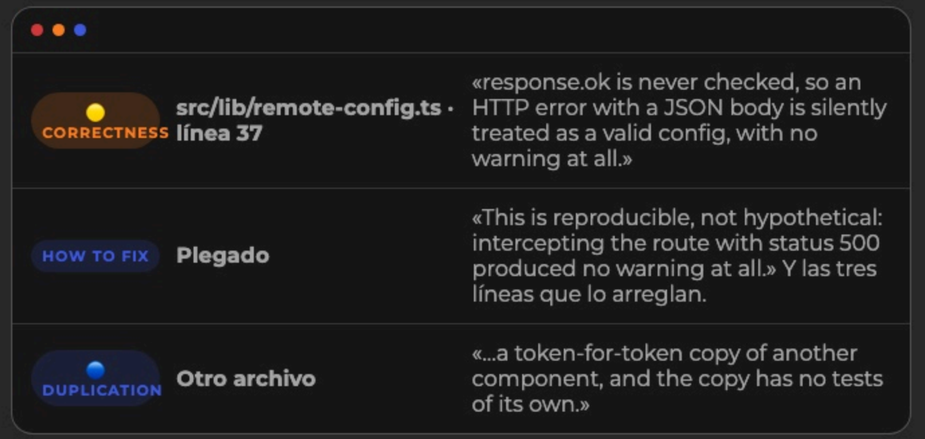
Si necesita un dato, escribe la query exacta y la corro yo. Funciona mejor de lo que suena, y no hay riesgo.

→ Demo: una sesión real, ya corrida, en Orca

FLUJO 2 · UN COMANDO `/pepito:review-pr acme/web-app#192`

Tres lanes, dos STOPs, y yo elijo.

LANE A	Código: pocos hallazgos, alta confianza.
LANE B	Seguridad: solo lo que se puede explotar de verdad.
LANE C	Funcional: un navegador prueba lo que el ticket promete, no solo el diff.
STOP 1	Checks rojos o sin review de otra IA: pregunta antes de seguir.
STOP 2	La lista fusionada, numerada. Yo elijo qué se publica; recién ahí comenta.



Comenta como una persona: corto, en la línea, en inglés, con mi cuenta. Nunca aprueba, nunca pide cambios, nunca mergea.

→ Ejemplo ilustrativo · en la charla: una review real, en Orca

EL PEPITO QUE ESCRIBE TICKETS · PEPITO-TICKET-MACHINE

Un ticket que un humano quiera leer.

01

Descripción corta

Contexto, alcance, criterios de aceptación y preguntas. Punteros y decisiones, no la investigación transcripta.

02

Un comentario plegado

«In-depth implementation details»: todo lo que necesita la IA que implementa, escondido para quien no lo necesita.

03

El gate de duplicados

Antes de crear: el síntoma, la superficie, el epic y los labels, y una búsqueda en los diarios. Ante la duda, no crear.

> **Un ticket que falta cuesta una corrida.** Un duplicado cuesta dos implementaciones.

CAPÍTULO 6

06

Iterar, siempre.

Nada de esto salió bien la primera vez. Cada corrección se escribe, y cada viernes el coder aprende. Con tus rutinas o tus agentes va a pasar lo mismo: hacete el tiempo para pulirlos.

CÓMO MEJORA · CADA VEZ

La clave: feedback, cada vez. Por escrito.

Claude hace algo → **Yo corrijo, y digo por qué** → **El librarian lo guarda** → **La próxima sesión arranca de ahí**

01

Una carpeta en git

Convenciones por cliente, gotchas, handoffs, el archivo del día. Los agentes la leen antes de arrancar: nunca explico dos veces lo mismo.

02

Siempre con el porqué

Cada corrección lleva su razón, así la regla tiene sentido después. Sin porqué, se rompe en el primer caso raro.

03

Hacete el tiempo

Cuando tengas tus rutinas o tus agentes, reservá un rato por semana para pulirlos. Cada vez funcionan mejor; sin eso, se degradan.

FLUJO 3 · CODER-REVIEW, VIERNES 17:30

Si va a codear, tiene que aprender.

EL PEDIDO

«Quiero irlo mejorando, así cuando programe cada vez lo haga mejor y a lo que nosotros necesitamos.»

Para la rutina de los viernes, el mismo día que nació el coder.

QUÉ MIRA	Los comentarios de la semana en las PRs y mis correcciones en las sesiones.
QUÉ HACE	Agrupa lo que se repite y lo escribe como regla en el coder, con backup. 26 reglas en ocho viernes.
EL MEJOR FEEDBACK	Es el mío: la tercera regla empezó como un comentario que dejé en la PR de una compañera.



LO QUE HOY NO SE NEGOCIA

Seis reglas que quedaron.

01

Un requisito, un ticket

Ninguna rutina escribe un ticket sola. Todo pasa por la ticket-machine y su gate.

02

Evidencia, nunca conclusiones

Un agente barato devuelve archivo, línea y snippet. Interpretar es de la sesión principal.

03

Scope discipline

Se entrega lo aprobado. Lo demás se nombra en una línea y lo decide el humano.

04

Una sola fuente

Estándares en el coder, formato en la ticket-machine, convenciones en un archivo. Lo que se repite en dos lugares se desincroniza.

05

Las preguntas no bloquean

Sección «Preguntas», mejor suposición, y seguir. Yo respondo después.

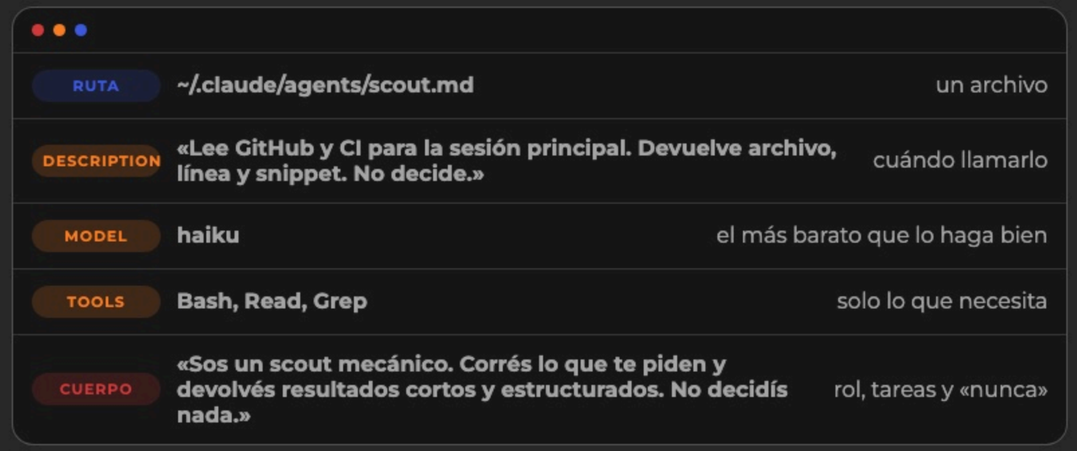
06

Todo en git

Agentes, skills, rutinas y base de conocimiento versionados. Cada cambio tiene historia.

CÓMO ARMAR EL TUYO

Un Pepito es un archivo de texto.



SUBAGENTE — Un `.md` en `~/.claude/agents/`: rol, modelo, herramientas y sus «nunca». Este es mi scout, tal cual. Cabe en una pantalla.

SKILL — Un `.md` en `~/.claude/skills/`: los pasos, y `$ARGUMENTS` para lo que va después de la barra. Así son implement-ticket y review-pr.

RUTINA — El mismo tipo de documento, con un horario en Orca. Si no escribís código, tu primer Pepito es este.

Cómo lo escribís: pedíselo a Claude («armame un subagente que haga X y nunca haga Y») y corregilo cada vez que falle.

CÓMO EMPEZAR

O usá los míos: el plugin del equipo.

QUÉ TRAE

Cuatro subagentes, siete skills

coder, scout, librarian y ticket-machine. implement-ticket, review-pr, create-ticket y las cuatro rutinas del día, listas para ponerles horario.

QUÉ NO TRAE

Nada mío, nada del cliente

Lo tuyo vive en tu máquina: un perfil con tu idioma, tu Jira y tus repos. El coder aprende de tus PRs, no de las mías.

CÓMO SIGUE

La historia completa

El día de la charla estaba en revisión. Hoy el equipo lo instala con un script, y cómo se armó está contado en el blog de Streaver.

Si no escribís código, el camino es el de la primera parte: describí la tarea en un párrafo, ponele un horario, que escriba a un archivo y que pregunte sin bloquear. El primer paso es el mismo: la rutina que te ahorre la primera media hora.

streaver.com/insights/blogs/claude-code-plugin-for-teams

DESDE EL 20 DE JULIO

Siete semanas después, en números.

39
días de trabajo registrados, uno por archivo.

116
reuniones archivadas con notas y action items.

32
tickets con handoff para retomar el trabajo.

8
viernes de review del coder, con reglas nuevas.

25
documentos de contexto por cliente y proyecto.

Todo vive en un repo git. Cada corrida deja huella, y la siguiente arranca desde ahí.

→ Demo: la carpeta que guarda el librarian, tal cual está hoy

LINKS · PARA DESPUÉS

Para profundizar.

ORCA	onorca.dev · orca.build/docs/model/worktrees · orca.build/docs/cli/automations
GIT WORKTREES	git-scm.com/docs/git-worktree · otra rama en otra carpeta, sin volver a clonar. Es de git, no de Orca.
CLAUDE CODE	code.claude.com/docs/en/subagents · .../skills · .../memory · .../hooks
CÓMO PENSAR LOS AGENTES	anthropic.com/engineering/building-effective-agents (orchestrator-workers) · claude.com/blog/how-anthropic-teams-use-claude-code
QUÉ AUTOMATIZAR	sre.google/sre-book/eliminating-toil · xkcd.com/1205
FASE 0	github.com/mattpocock/skills · el workflow del que copié la Fase 0 de implement-ticket

Empezó con un chat. Puede empezar con el tuyo.

Action item para los que no tienen ninguna automatización corriendo: elegí una tarea que hacés todos los días, escribímela, y en 30 minutos la dejamos automatizada con Claude y Orca.



Lo que no automatizo: aprobar o mergear una PR, y publicar algo con mi nombre sin haberlo leído. Eso sigue siendo mío. Los Pepitos proponen; yo decido.

streaver

Escribime por LinkedIn ¿Preguntas?